

Lambda Expression vs Method Reference in Java

The **basic difference** is:

A lambda expression defines an implementation of a functional interface, whereas a method reference reuses an existing method as that implementation.

In other words:

- **Lambda:** *"Here is the code that should execute."*
- **Method reference:** *"Use this already-existing method."*

1. Basic comparison

Parameter	Lambda Expression	Method Reference
Definition	A lambda expression is an anonymous function that provides an implementation for a functional interface .	A method reference is a shorthand syntax for referring to an existing method or constructor .
Introduced	Java 8	Java 8
Purpose	Used to write behavior directly.	Used to reuse existing behavior.
Syntax	(parameters) -> expression or (parameters) -> { statements; }	ClassName::methodName, object::methodName, or ClassName::new
Creates behavior	Defines the behavior inline.	Points to behavior that already exists.
Requires functional interface?	Yes, in normal Java usage.	Yes, it is used where a functional-interface target is expected.
Code size	Usually longer.	Usually shorter.
Readability	Better when custom logic is required.	Better when an existing method exactly matches the required functional-interface method.
Reuses existing method?	Not necessarily.	Yes.
Can contain multiple statements?	Yes, using { }.	No; it simply refers to an existing method/constructor.

Can contain custom logic?	Yes.	No; the referenced method contains the logic.
Parameters	Lambda explicitly declares/receives parameters.	Parameters are implicitly passed to the referenced method.
Return value	Can explicitly calculate and return a value.	Uses the return value of the referenced method.
Best use	When implementation logic is specific to that location.	When an existing method already provides exactly the required behavior.

2. Lambda expression

Definition

A **lambda expression** is an anonymous function that can be used to provide an implementation of a **functional interface**.

General syntax

(parameters) -> expression

or

```
(parameters) -> {
    statements;
}
```

Example

```
Function<Integer, Integer> square = x -> x * x;
```

```
System.out.println(square.apply(5));
```

Output:

25

Here:

```
x -> x * x
```

is a lambda expression.

It says:

Take x as input and return $x * x$.

3. Method reference

Definition

A **method reference** is a shorthand notation for referring to an existing method instead of writing a lambda expression that simply calls that method.

General syntax

`ClassName::methodName`

Example:

```
Function<String, Integer> converter = Integer::parseInt;  
  
System.out.println(converter.apply("100"));
```

Here:

`Integer::parseInt`

is a method reference.

It means:

When the functional interface needs to execute, use the existing `Integer.parseInt()` method.

4. The most important relationship

A method reference is essentially a **more concise alternative to certain lambda expressions**.

For example:

Lambda

```
Consumer<String> printer = name -> System.out.println(name);
```

Method reference

```
Consumer<String> printer = System.out::println;
```

Both perform the same operation.

The lambda says:

Take name → call `System.out.println(name)`

The method reference says:

Use `System.out.println`

So:

Lambda

↓

provides implementation

Method Reference

↓

refers to existing implementation

5. Direct comparison with examples

Lambda Expression	Equivalent Method Reference	Explanation
<code>x -> System.out.println(x)</code>	<code>System.out::println</code>	Calls <code>println()</code>
<code>x -> Integer.parseInt(x)</code>	<code>Integer::parseInt</code>	Calls <code>parseInt()</code>
<code>x -> x.toUpperCase()</code>	<code>String::toUpperCase</code>	Calls <code>toUpperCase()</code>
<code>(a, b) -> a.compareTo(b)</code>	<code>String::compareTo</code>	Calls <code>compareTo()</code>
<code>x -> Math.abs(x)</code>	<code>Math::abs</code>	Calls <code>Math.abs()</code>
<code>x -> new String(x)</code>	<code>String::new</code>	Calls the constructor
<code>x -> x.length()</code>	<code>String::length</code>	Calls <code>length()</code>

The important point is that **not every lambda can be converted into a method reference**.

6. Example: Lambda and method reference with `forEach()`

Suppose we have:

```
List<String> names = List.of("Anurag", "Rahul", "Amit");
```

Using lambda

```
names.forEach(name -> System.out.println(name));
```

Using method reference

```
names.forEach(System.out::println);
```

Both produce:

```
Anurag
Rahul
Amit
```

Difference

Lambda	Method Reference
name -> System.out.println(name)	System.out::println
Explicitly describes the operation.	Directly references the existing operation.
More verbose.	More concise.
Useful if additional processing is required.	Useful when directly delegating to an existing method.

7. When lambda is better

Suppose you want to perform additional processing.

```
names.forEach(name -> {
    String upper = name.toUpperCase();
    System.out.println("Name = " + upper);
});
```

You cannot simply replace this entire logic with:

```
System.out::println
```

because System.out::println does not perform the toUpperCase() operation.

A method reference is appropriate only when an existing method already represents the required operation.

8. Four types of method references

Java provides four major forms of method/constructor references.

Type	Syntax	Example	Equivalent Lambda
1. Static method reference	ClassName::staticMethod	Integer::parseInt	x -> Integer.parseInt(x)
2. Instance method of a particular object	object::instanceMethod	System.out::println	x -> System.out.println(x)
3. Instance method of an arbitrary object of a particular type	ClassName::instanceMethod	String::toUpperCase	x -> x.toUpperCase()
4. Constructor reference	ClassName::new	ArrayList::new	() -> new ArrayList<>()

9. Static method reference

Existing method

```
public static int square(int x) {  
    return x * x;  
}
```

Lambda

```
Function<Integer, Integer> f = x -> Math.abs(x);
```

Method reference

```
Function<Integer, Integer> f = Math::abs;
```

Here:

Math::abs

refers to the existing static method:

```
Math.abs()
```

10. Instance method reference

Consider:

```
String name = "Anurag";
```

Lambda

```
Supplier<String> s = () -> name.toUpperCase();
```

Method reference

```
Supplier<String> s = name::toUpperCase;
```

Here:

```
name::toUpperCase
```

means:

Invoke `toUpperCase()` on this particular name object.

11. Instance method of an arbitrary object

This is an important distinction.

Consider:

```
List<String> names = List.of("Rahul", "Anurag", "Amit");
```

Lambda

```
names.stream()  
    .map(name -> name.toUpperCase())  
    .forEach(System.out::println);
```

The lambda:

```
name -> name.toUpperCase()
```

can be replaced with:

```
String::toUpperCase
```

Therefore:

```
names.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Here `String::toUpperCase` means:

For each `String` object supplied by the stream, invoke its `toUpperCase()` method.

12. Constructor reference

A constructor can also be referenced.

Lambda

```
Supplier<ArrayList<String>> supplier =
    () -> new ArrayList<>();
```

Constructor reference

```
Supplier<ArrayList<String>> supplier =
    ArrayList::new;
```

Both mean:

Create a new `ArrayList` object.

Another example:

```
Function<String, StringBuilder> creator =
    StringBuilder::new;
```

Equivalent lambda:

```
Function<String, StringBuilder> creator =
    text -> new StringBuilder(text);
```

13. Underlying conceptual difference

This is the most important part.

Consider:

```
x -> x.toUpperCase()
```

The lambda explicitly describes an operation:

Input x
↓
call x.toUpperCase()
↓
return result

The method reference:

String::toUpperCase

does not explicitly declare x.

Instead, Java determines how the method reference maps to the target functional interface.

For example:

```
Function<String, String> f = String::toUpperCase;
```

The compiler understands this approximately as:

```
Function<String, String> f =  
    x -> x.toUpperCase();
```

Therefore, the method reference is essentially a **compact representation of an existing method invocation**.

14. Lambda can contain additional logic

This is one of the strongest differences.

Lambda

```
Function<String, String> f = name -> {  
    String result = name.trim();  
    result = result.toUpperCase();  
    return "USER: " + result;  
};
```

This contains multiple operations.

A method reference cannot directly express this:

```
// Not possible  
String::trim.toUpperCase
```

Instead, use a lambda:

```
name -> "USER: " + name.trim().toUpperCase()
```

15. Method reference does not mean "method execution"

This is a common interview question.

Consider:

```
System.out::println
```

This **does not immediately execute** `println()`.

It creates a reference to the method that can later be invoked through a functional interface.

For example:

```
Consumer<String> printer = System.out::println;
```

The method executes when:

```
printer.accept("Hello");
```

is called.

Therefore:

Expression	Meaning
<code>System.out.println("Hello")</code>	Immediately invokes the method
<code>System.out::println</code>	Creates a method reference
<code>printer.accept("Hello")</code>	Invokes the referenced method

16. Lambda vs method reference: execution perspective

Stage	Lambda	Method Reference
Source	<code>x -> x.toUpperCase()</code>	<code>String::toUpperCase</code>
What is provided?	A lambda implementation	A reference to an existing method
Target	Functional interface	Functional interface
Execution	When functional-interface	When functional-interface method

	method is invoked	is invoked
Immediate execution?	No	No
Existing method required?	No	Yes

17. Functional interface connection

Both are strongly connected to functional interfaces.

For example:

```
@FunctionalInterface
interface Converter {
    String convert(String value);
}
```

Lambda

```
Converter c = value -> value.toUpperCase();
```

Method reference

```
Converter c = String::toUpperCase;
```

The functional interface provides the **target shape**:

```
String convert(String value)
```

Java then checks whether the lambda or method reference is compatible with that shape.

18. Parameter handling

Lambda	Method Reference
Parameters are explicitly represented in the lambda.	Parameters are inferred from the target functional interface and method signature.
x -> x.length()	String::length
(a, b) -> a.compareTo(b)	String::compareTo
x -> Integer.parseInt(x)	Integer::parseInt

For example:

```
Function<String, Integer> f = String::length;
```

The functional interface says:

$\text{String} \rightarrow \text{Integer}$

Therefore Java understands:

```
x -> x.length()
```

19. Important limitation: not every lambda has a method-reference equivalent

Consider:

```
Function<Integer, Integer> f =  
    x -> x * 2;
```

There isn't an existing method necessarily corresponding to:

```
x * 2
```

So you normally use the lambda.

Another example:

```
Predicate<String> p =  
    s -> s.length() > 5;
```

The logic combines:

```
length()  
+  
comparison > 5
```

There is no direct single existing method representing the entire operation.

Therefore a lambda is appropriate.

20. Practical decision rule

Situation	Prefer
You need custom logic	Lambda

You need multiple statements	Lambda
You need conditions/calculations	Lambda
An existing method exactly performs the required operation	Method reference
You want concise code	Method reference
You want to improve readability by reusing a named method	Method reference
You need a constructor as the functional behavior	Constructor reference

21. Final summary

Aspect	Lambda Expression	Method Reference
Core idea	Define behavior	Reuse existing behavior
Meaning	"Execute this logic"	"Use this existing method"
Syntax	<code>x -> x.toUpperCase()</code>	<code>String::toUpperCase</code>
Existing method needed?	No	Yes
Custom logic	Yes	No
Multiple statements	Yes	No
Conciseness	Moderate	High
Readability	Good for custom operations	Excellent for simple delegation
Functional interface	Required as target	Required as target
Parameter declaration	Explicit/inferred lambda parameters	Determined by target type and referenced method
Execution	Delayed until functional-interface method is invoked	Delayed until functional-interface method is invoked
Method execution	Lambda body executes	Referenced method executes later

	later	
Constructor support	<code>() -> new Employee()</code>	<code>Employee::new</code>
Best description	Implementation	Reference to implementation

One-line interview answer

A lambda expression provides the implementation of a functional interface directly, while a method reference provides a concise reference to an already-existing method or constructor whose signature is compatible with that functional interface.

Easy way to remember

Lambda:

`x -> x.toUpperCase()`

└ "Here is the logic."

Method Reference:

`String::toUpperCase`

└ "Use this existing method."

So the **underlying difference is not that they are two completely different mechanisms**—both are ways of supplying behavior to a functional interface. The key difference is **how that behavior is supplied: lambda defines it inline; method reference delegates to an existing method/constructor.**

Yes. Lambda expressions and method references can be used with static methods, non-static (instance) methods, static nested classes, non-static inner classes, local classes, and anonymous classes, provided the lambda/method reference has a compatible functional-interface target type.

The key point is:

A lambda expression does not directly belong to a class. It implements a functional interface. A method reference points to an existing compatible method or constructor.

1. Basic concept

Concept	Lambda Expression	Method Reference
Definition	Anonymous implementation of a functional interface.	Shorthand for referring to an existing method or constructor.
Main purpose	Define behavior inline.	Reuse existing behavior.
Syntax	(parameters) -> expression	ClassName::method, object::method, or ClassName::new
Works with static methods?	Yes	Yes
Works with instance methods?	Yes	Yes
Works with static nested classes?	Yes	Yes
Works with non-static inner classes?	Yes	Yes
Works inside local classes?	Yes	Yes
Works inside anonymous classes?	Yes	Yes
Can reference constructors?	Yes, using new inside lambda	Yes, using ::new
Requires functional interface target?	Yes	Yes

2. First important distinction: static vs non-static

Consider:

```
class Calculator {  
  
    static int add(int a, int b) {  
        return a + b;  
    }  
  
    int multiply(int a, int b) {  
        return a * b;  
    }  
}
```

There are two different methods:

Method	Type	Invocation
add()	Static	Calculator.add(10, 20)
multiply()	Instance/non-static	calculator.multiply(10, 20)

Both can be used with lambdas and method references.

3. Static method with lambda

Suppose:

```
@FunctionalInterface  
interface Operation {  
    int calculate(int a, int b);  
}
```

We can use a lambda:

```
Operation operation =  
    (a, b) -> Calculator.add(a, b);
```

Usage:

```
System.out.println(operation.calculate(10, 20));
```

Output:

30

The lambda says:

Receive a and b, then call Calculator.add(a, b).

4. Static method with method reference

The same operation can be written more concisely:

Operation operation = Calculator::add;

Complete example:

```
class Calculator {  
  
    static int add(int a, int b) {  
        return a + b;  
    }  
}  
  
@FunctionalInterface  
interface Operation {  
    int calculate(int a, int b);  
}  
  
public class Demo {  
  
    public static void main(String[] args) {  
  
        Operation operation = Calculator::add;  
  
        System.out.println(operation.calculate(10, 20));  
    }  
}
```

Output:

30

Comparison

Lambda	Method Reference
(a, b) -> Calculator.add(a, b)	Calculator::add
Explicitly calls the static method.	Directly references the static method.

More verbose.	More concise.

5. Static method: underlying mapping

The following:

```
Operation operation = Calculator::add;
```

is conceptually equivalent to:

```
Operation operation =  
    (a, b) -> Calculator.add(a, b);
```

Therefore:

```
Calculator::add  
  ↓  
Calculator.add(a, b)  
  ↓  
Operation.calculate(a, b)
```

The method reference does **not execute add() immediately**.

It provides a reference that can be invoked later through `Operation.calculate()`.

6. Non-static/instance method with lambda

Now consider:

```
class Calculator {  
  
    int multiply(int a, int b) {  
        return a * b;  
    }  
}
```

Create an object:

```
Calculator calculator = new Calculator();
```

Using a lambda:

```
Operation operation =
```

```
(a, b) -> calculator.multiply(a, b);
```

Then:

```
System.out.println(operation.calculate(10, 20));
```

Output:

```
200
```

7. Non-static/instance method with method reference

The same code can be written as:

```
Calculator calculator = new Calculator();
```

```
Operation operation =  
    calculator::multiply;
```

Complete example:

```
class Calculator {  
  
    int multiply(int a, int b) {  
        return a * b;  
    }  
}  
  
@FunctionalInterface  
interface Operation {  
    int calculate(int a, int b);  
}  
  
public class Demo {  
  
    public static void main(String[] args) {  
  
        Calculator calculator = new Calculator();  
  
        Operation operation = calculator::multiply;  
  
        System.out.println(operation.calculate(10, 20));  
    }  
}
```

Output:

```
200
```

Important difference

Static method	Instance method
Calculator::add	calculator::multiply
No object required	An object is required
References a static method	References a method belonging to a particular object
ClassName::method	object::method

8. Static method vs instance method

Feature	Static Method	Instance Method
Belongs to	Class	Object
Object required to invoke?	No	Yes
Lambda example	(a,b) -> Calculator.add(a,b)	(a,b) -> calculator.multiply(a,b)
Method reference	Calculator::add	calculator::multiply
General form	ClassName::staticMethod	object::instanceMethod

9. Instance method of an arbitrary object

There is another important method-reference form.

Consider:

```
class Employee {  
    String getName() {  
        return "Anurag";  
    }  
}
```

Suppose:

```
Function<Employee, String> function =  
    employee -> employee.getName();
```

This can become:

```
Function<Employee, String> function =  
    Employee::getName;
```

Notice that we didn't write:

```
employee::getName
```

Instead:

```
Employee::getName
```

is used.

This means:

For whichever Employee object is supplied as the argument, call getName() on that object.

10. Three important method-reference forms

Type	Syntax	Example	Equivalent lambda
Static method	ClassName::staticMethod	Math::abs	x -> Math.abs(x)
Instance method of a particular object	object::instanceMethod	System.out::println	x -> System.out.println(x)
Instance method of an arbitrary object	ClassName::instanceMethod	String::toUpperCase	s -> s.toUpperCase()
Constructor	ClassName::new	ArrayList::new	() -> new ArrayList<>()

11. Static nested class

A **static nested class** is a class declared with the static keyword inside another class.

Example:

```
class Outer {  
    static class Inner {
```

```

    static int add(int a, int b) {
        return a + b;
    }

    int multiply(int a, int b) {
        return a * b;
    }
}

```

The nested class can contain both static and instance methods.

12. Lambda with static nested class

```

Operation operation =
    (a, b) -> Outer.Inner.add(a, b);

```

Example:

```

class Outer {

    static class Inner {

        static int add(int a, int b) {
            return a + b;
        }
    }
}

```

Then:

```

Operation operation =
    (a, b) -> Outer.Inner.add(a, b);

```

```

System.out.println(operation.calculate(10, 20));

```

Output:

30

13. Method reference with static nested class

The lambda:

```

(a, b) -> Outer.Inner.add(a, b)

```

can be replaced with:

Outer.Inner::add

Example:

Operation operation = Outer.Inner::add;

So:

Lambda	Method Reference
(a,b) -> Outer.Inner.add(a,b)	Outer.Inner::add

The static nested class does **not prevent** either feature from being used.

14. Instance method of a static nested class

Suppose:

```
class Outer {  
    static class Inner {  
        int multiply(int a, int b) {  
            return a * b;  
        }  
    }  
}
```

Create the object:

Outer.Inner inner = new Outer.Inner();

Lambda

Operation operation =
 (a, b) -> inner.multiply(a, b);

Method reference

Operation operation =
 inner::multiply;

Therefore:

Static nested class



static method instance method



Lambda / Method Reference

15. Non-static inner class

A **non-static inner class** is a class declared inside another class without the static keyword.

Example:

```
class Outer {  
    class Inner {  
        int add(int a, int b) {  
            return a + b;  
        }  
    }  
}
```

To create the inner-class object:

```
Outer outer = new Outer();
```

```
Outer.Inner inner = outer.new Inner();
```

16. Lambda with non-static inner class

```
Operation operation =  
    (a, b) -> inner.add(a, b);
```

Example:

```
class Outer {  
    class Inner {  
        int add(int a, int b) {  
            return a + b;  
        }  
    }  
}
```

Then:

```
Outer outer = new Outer();  
Outer.Inner inner = outer.new Inner();
```

```
Operation operation =  
    (a, b) -> inner.add(a, b);
```

```
System.out.println(operation.calculate(10, 20));
```

Output:

30

17. Method reference with non-static inner class

The same code can be written as:

```
Operation operation = inner::add;
```

Therefore:

Lambda	Method Reference
<code>(a,b) -> inner.add(a,b)</code>	<code>inner::add</code>

The fact that Inner is a **non-static inner class** does not prevent method references from being used.

18. Lambda inside an inner class

A lambda can also be **declared inside** an inner class.

Example:

```
class Outer {  
    class Inner {  
        void process() {  
            Consumer<String> consumer =  
                name -> System.out.println(name);  
        }  
    }  
}
```

```

        consumer.accept("Anurag");
    }
}

```

This is perfectly valid.

A method reference can also be used:

```

class Outer {

    class Inner {

        void process() {

            Consumer<String> consumer =
                System.out::println;

            consumer.accept("Anurag");
        }
    }
}

```

19. Inner-class this and lambda

This is an important difference when working with nested classes.

Consider:

```

class Outer {

    String name = "Outer";

    class Inner {

        String name = "Inner";

        void display() {

            Runnable r = () -> {
                System.out.println(this.name);
            };

            r.run();
        }
    }
}

```

Output:

Inner

The this inside a lambda refers to the **enclosing instance**, not a separate lambda object.

In this example:

this.name

refers to:

Inner.this.name

20. Lambda vs anonymous class with this

This is an important underlying distinction.

Lambda	Anonymous Class
Does not create a new this context.	Creates its own object/context.
this refers to the enclosing object.	this refers to the anonymous-class object.
Designed primarily for functional interfaces.	Can implement interfaces or extend classes.

Example lambda:

```
Runnable r = () -> {  
    System.out.println(this);  
};
```

The this refers to the enclosing class instance.

21. Method references and inner-class methods

Suppose:

```
class Outer {  
    class Inner {  
        void print(String message) {
```

```

        System.out.println(message);
    }

    void process() {

        Consumer<String> consumer =
            this::print;

        consumer.accept("Hello");
    }
}

```

Here:

`this::print`

is a method reference to the current Inner object's `print()` method.

Equivalent lambda:

```

Consumer<String> consumer =
    message -> this.print(message);

```

So:

Lambda	Method Reference
<code>message -> this.print(message)</code>	<code>this::print</code>

22. Static method inside an inner/nested class

Suppose:

```

class Outer {

    static class Inner {

        static void print(String message) {
            System.out.println(message);
        }
    }
}

```

Method reference:

```

Consumer<String> consumer =

```

```
Outer.Inner::print;
```

Equivalent lambda:

```
Consumer<String> consumer =  
    message -> Outer.Inner.print(message);
```

23. Constructor references with nested classes

Method references are also capable of referencing constructors.

Static nested class

```
class Outer {  
  
    static class Employee {  
    }  
}
```

Constructor reference:

```
Supplier<Outer.Employee> supplier =  
    Outer.Employee::new;
```

Equivalent lambda:

```
Supplier<Outer.Employee> supplier =  
    () -> new Outer.Employee();
```

24. Non-static inner-class constructor

For a non-static inner class:

```
class Outer {  
  
    class Inner {  
    }  
}
```

An inner-class object normally requires an enclosing Outer instance.

For example:

```
Outer outer = new Outer();
```

```
Supplier<Outer.Inner> supplier =
```

```
outer::newInner;
```

But Java does **not** provide a normal `outer::newInner` syntax automatically for an arbitrary inner-class constructor because `Inner`'s constructor is tied to the enclosing `Outer` instance.

A lambda is often used:

```
Supplier<Outer.Inner> supplier =  
    () -> outer.new Inner();
```

This illustrates an important point:

Not every constructor lambda has a directly corresponding constructor-reference syntax, especially when an enclosing instance is required.

25. Local classes

A **local class** is a class declared inside a method, constructor, or block.

Example:

```
void process() {  
    class Calculator {  
        int add(int a, int b) {  
            return a + b;  
        }  
    }  
  
    Calculator calculator = new Calculator();  
  
    Operation operation =  
        (a, b) -> calculator.add(a, b);  
}
```

A method reference can also be used:

```
Operation operation =  
    calculator::add;
```

Therefore local classes can also participate in lambda/method-reference usage.

26. Anonymous classes

Lambda expressions and method references can be used **inside** anonymous classes.

Example:

```
Runnable r = new Runnable() {  
  
    @Override  
    public void run() {  
  
        Consumer<String> consumer =  
            message -> System.out.println(message);  
  
        consumer.accept("Hello");  
    }  
};
```

Method reference:

```
Runnable r = new Runnable() {  
  
    @Override  
    public void run() {  
  
        Consumer<String> consumer =  
            System.out::println;  
  
        consumer.accept("Hello");  
    }  
};
```

27. Can an anonymous class itself be replaced by a lambda?

Sometimes.

If an anonymous class implements a **functional interface**, it can often be replaced by a lambda.

Anonymous class

```
Runnable r = new Runnable() {  
  
    @Override  
    public void run() {  
        System.out.println("Hello");  
    }  
};
```

Lambda

```
Runnable r =  
    () -> System.out.println("Hello");
```

However, an anonymous class and a lambda are **not universally interchangeable**.

A lambda requires a functional interface.

An anonymous class can implement an interface with multiple abstract methods or extend a class.

28. Complete comparison: class types

Class/Method Type	Lambda	Method Reference	Example
Static method	✓	✓	Calculator::add
Instance method	✓	✓	calculator::multiply
Static nested class	✓	✓	Outer.Inner::add
Instance method of static nested class	✓	✓	inner::multiply
Non-static inner class	✓	✓	inner::add
Method inside non-static inner class	✓	✓	this::print
Local class	✓	✓	calculator::add
Anonymous class	✓ inside it	✓ inside it	System.out::println
Static method of outer class	✓	✓	Outer::method
Instance method of outer class	✓	✓	outer::method
Constructor of normal class	✓	✓	() -> new Employee() / Employee::new
Constructor of static nested class	✓	✓	Outer.Inner::new
Constructor requiring enclosing instance	✓	Sometimes not directly	() -> outer.new Inner()

--	--	--	--

29. Very important: lambda is not static or non-static

A common misconception is:

"Can I make a lambda static or non-static?"

Not exactly.

A lambda expression itself is **not classified as a static method or instance method**.

Instead, it provides an implementation for a functional interface.

For example:

```
Function<Integer, Integer> f =  
    x -> x * 2;
```

The lambda:

```
x -> x * 2
```

is neither:

static lambda

nor:

non-static lambda

It is simply a lambda expression.

30. The same applies to method references

A method reference **does have a target method**, and that target can be static or instance-based.

Static

Math::abs

Particular object

calculator::multiply

Arbitrary object of a type

String::toUpperCase

So the classification belongs to the **referenced method**, not to the method-reference expression itself.

31. Complete example

```
import java.util.function.*;

class Outer {

    static int staticAdd(int a, int b) {
        return a + b;
    }

    int instanceSubtract(int a, int b) {
        return a - b;
    }

    static class StaticNested {

        static int staticMultiply(int a, int b) {
            return a * b;
        }

        int instanceDivide(int a, int b) {
            return a / b;
        }
    }

    class Inner {

        int instanceAdd(int a, int b) {
            return a + b;
        }

        void process() {

            Operation lambda =
                (a, b) -> instanceAdd(a, b);

            Operation methodReference =
                this::instanceAdd;

            System.out.println(
```

```

        lambda.calculate(10, 20));

    System.out.println(
        methodReference.calculate(10, 20));
    }
}
}

```

```

@FunctionalInterface
interface Operation {

    int calculate(int a, int b);
}

```

Here all of the following concepts are possible:

```

Outer.staticAdd
Outer.instanceSubtract
Outer.StaticNested.staticMultiply
Outer.StaticNested.instanceDivide
Outer.Inner.instanceAdd

```

and they can be used as lambda targets or method references when their signatures match the functional interface.

32. Final interview-oriented comparison

Question	Answer
Can a lambda call a static method?	Yes
Can a lambda call an instance method?	Yes
Can a method reference reference a static method?	Yes
Can a method reference reference an instance method?	Yes
Can lambda expressions be used inside static nested classes?	Yes
Can lambda expressions be used inside non-static inner classes?	Yes
Can method references be used inside static nested classes?	Yes

Can method references be used inside non-static inner classes?	Yes
Can this::method be used?	Yes , for an instance method of the current enclosing object.
Can ClassName::staticMethod be used?	Yes
Can object::instanceMethod be used?	Yes
Can ClassName::instanceMethod be used?	Yes , when the first functional-interface parameter supplies the receiver object.
Can constructors be referenced?	Yes , using ClassName::new, subject to constructor/context compatibility.
Is a lambda itself static?	No
Is a lambda itself non-static?	No
Is a method reference itself static?	No ; it refers to a static or instance method depending on its form.

Final takeaway

Lambda	Method Reference
Defines behavior	References existing behavior
(a, b) -> Calculator.add(a, b)	Calculator::add
(a, b) -> calculator.multiply(a, b)	calculator::multiply
(s) -> s.toUpperCase()	String::toUpperCase
(message) -> this.print(message)	this::print
() -> new Employee()	Employee::new

The simplest rule to remember

STATIC METHOD

↓

ClassName::methodName

INSTANCE METHOD OF A PARTICULAR OBJECT

↓
object::methodName

INSTANCE METHOD OF ANY OBJECT OF THAT TYPE
↓
ClassName::methodName

CONSTRUCTOR
↓
ClassName::new

CUSTOM / MULTI-STEP LOGIC
↓
Lambda expression

Therefore, static classes, non-static inner classes, local classes, and anonymous classes do not fundamentally restrict lambda expressions or method references. The decisive requirement is that the lambda/method reference must be compatible with the target functional interface.

Overloading and Overriding in Lambda Expressions, Method References, Inner Classes, and Exceptions

The most important point is that **overloading and overriding do not apply in exactly the same way to all four concepts.**

- **Overloading** = same method name, different parameter list, usually within the same class/type.
- **Overriding** = subclass/implementing class provides a new implementation of an inherited/implemented instance method with a compatible signature.
- A **lambda expression itself cannot be overloaded or overridden**, because it is not a named method.
- A **method reference itself cannot be overloaded or overridden**, because it is a reference to an existing method/constructor. However, **overloaded methods can be selected by a method reference.**
- **Inner-class methods can both be overloaded and overridden.**
- **Exceptions affect overriding strongly**, especially checked exceptions.

1. Basic definitions

Concept	Definition	Simple example
Overloading	Defining multiple methods with the same name but different parameter lists.	<code>add(int,int)</code> and <code>add(double,double)</code>
Overriding	A subclass provides its own implementation of an inherited instance method with a compatible signature.	<code>Dog.sound()</code> overriding <code>Animal.sound()</code>
Lambda expression	An anonymous implementation of a functional interface.	<code>x -> x * 2</code>
Method reference	A compact reference to an existing method or constructor.	<code>Math::abs</code>
Inner class	A class declared inside another class.	<code>class Outer { class Inner {} }</code>
Checked exception	Exception that must be caught or declared.	<code>IOException</code>

Unchecked exception	Runtime exception that does not have to be declared or caught.	NullPointerException
----------------------------	--	----------------------

2. Can lambda expressions be overloaded or overridden?

Question	Answer	Explanation
Can a lambda itself be overloaded?	No	A lambda is not a named method.
Can a lambda itself be overridden?	No	A lambda does not define a class method that can be overridden.
Can overloaded functional-interface methods accept lambdas?	Yes	Java can select an overloaded method based on the lambda's target type, but ambiguity can occur.
Can a lambda implement an overridden interface method?	Yes	A lambda can provide the implementation of the single abstract method of a functional interface.
Can a lambda contain overloaded method calls?	Yes	Normal Java overload resolution applies inside the lambda body.
Can a lambda call an overridden method?	Yes	Normal polymorphism applies when the lambda executes the method.

3. Lambda implementing an interface method

Consider:

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
```

A lambda provides the implementation:

```
Calculator c = (a, b) -> a + b;
```

Here:

calculate()

↑
|

lambda provides implementation

But the lambda itself is **not a method named calculate()** that you can overload or override.

4. Lambda and overloaded methods

This is where overloading becomes particularly interesting.

```
static void process(Function<String, String> function) {  
    System.out.println("Function");  
}
```

```
static void process(UnaryOperator<String> operator) {  
    System.out.println("UnaryOperator");  
}
```

Now:

```
process(x -> x.toUpperCase());
```

may be **ambiguous**, because:

Function<String,String>

and

UnaryOperator<String>

are both compatible with the lambda.

Therefore Java may not know which overloaded process() method you mean.

Solution: cast the lambda

```
process((Function<String, String>)  
    x -> x.toUpperCase());
```

Now Java knows which overload to select.

5. Lambda and overloading: important rule

Situation	Result
Overloaded methods have clearly different parameter types	Compiler may select the correct overload.
Two functional-interface overloads are both compatible with the lambda	Ambiguous method call may occur.
Explicit lambda cast is provided	Ambiguity can be resolved.
Lambda parameter types are explicitly specified	May help overload resolution.
Lambda body contains incompatible operations for one overload	That overload may be eliminated.

6. Example: overloaded methods with different functional interfaces

```
interface StringProcessor {
    String process(String value);
}
```

```
interface NumberProcessor {
    int process(int value);
}
```

Suppose:

```
static void execute(StringProcessor processor) {
    System.out.println("String processor");
}
```

```
static void execute(NumberProcessor processor) {
    System.out.println("Number processor");
}
```

Then:

```
execute(x -> x.toUpperCase());
```

The compiler can infer that x should be a String because:

```
x.toUpperCase()
```

is applicable to String.

But with:

```
execute(x -> x);
```

the compiler may have insufficient information to determine the intended functional interface.

Explicit typing can help:

```
execute((String x) -> x);
```

7. Lambda and overriding

A lambda can implement an **overridden interface method**.

Suppose:

```
@FunctionalInterface
interface Animal {
    void sound();
}
```

A traditional implementation might be:

```
class Dog implements Animal {

    @Override
    public void sound() {
        System.out.println("Bark");
    }
}
```

Using a lambda:

```
Animal dog = () -> System.out.println("Bark");
```

The lambda provides the implementation of:

```
sound()
```

But technically, you should not say:

"The lambda overrides sound()."

A more accurate statement is:

The lambda provides the implementation required by the functional interface's abstract method.

8. Lambda vs traditional overriding

Traditional implementation	Lambda
class Dog implements Animal	Animal dog = () -> ...
Contains a named method sound()	Does not contain a named method declaration in source code
Can explicitly use @Override	Cannot write @Override on the lambda
Class participates in inheritance	Lambda does not create a normal named subclass in source code
Can have multiple methods	Functional interface must have one abstract method

For example, this is invalid:

```
Animal dog = new Animal() {  
    @Override  
    public void sound() {  
        System.out.println("Bark");  
    }  
};
```

The anonymous class explicitly overrides sound().

But:

```
Animal dog =  
    () -> System.out.println("Bark");
```

is a lambda implementation of the functional interface.

9. Method references and overloading

Method references are particularly interesting with overloaded methods.

Consider:

```
class Printer {  
    static void print(String value) {
```

```

    System.out.println("String: " + value);
}

static void print(Integer value) {
    System.out.println("Integer: " + value);
}
}

```

There are two overloaded methods:

```

print(String)
print(Integer)

```

Now:

```
Consumer<String> c = Printer::print;
```

selects:

```
print(String)
```

because Consumer<String> requires:

```
void accept(String)
```

Similarly:

```
Consumer<Integer> c = Printer::print;
```

selects:

```
print(Integer)
```

10. Method reference and overload resolution

Target functional interface	Method reference	Selected method
Consumer<String>	Printer::print	print(String)
Consumer<Integer>	Printer::print	print(Integer)
Function<String,Integer>	Parser::parse	Compatible parse(String)
Function<String,Double>	Parser::parse	Compatible parse(String) returning Double, if available
No target type	Printer::print	May be impossible/ambiguous

Key concept

A method reference by itself may not contain enough information to determine which overloaded method should be selected.

The **target functional interface** provides contextual type information.

11. Method reference and overriding

Suppose:

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

Now:

```
Dog dog = new Dog();
```

```
Runnable r = dog::sound;
```

When:

```
r.run();
```

executes, the overridden `Dog.sound()` is called.

So the method reference can participate in normal **runtime polymorphism**.

12. Method reference to an overridden method

```
Animal animal = new Dog();
```

```
Runnable r = animal::sound;
```

```
r.run();
```

Output:

Bark

Why?

Because:

```
animal  
↓  
actual object = Dog  
↓  
sound()  
↓  
Dog.sound()
```

The method reference does not disable Java's normal dynamic method dispatch.

13. Important distinction: ClassName::method and overriding

Consider:

```
class Animal {  
    void sound() {  
        System.out.println("Animal");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog");  
    }  
}
```

This:

```
Dog dog = new Dog();
```

```
Runnable r = dog::sound;
```

references the instance method through a particular object.

But a method reference such as:

`Animal::sound`

has different semantics: it is the **unbound instance-method reference** form and can be compatible with a functional interface whose first argument supplies the receiver object.

For example:

```
Consumer<Animal> c = Animal::sound;
```

Then:

```
c.accept(new Dog());
```

will invoke the overridden `Dog.sound()` because normal virtual dispatch applies.

14. Inner classes: overloading

Unlike lambdas, **methods inside inner classes can absolutely be overloaded.**

Example:

```
class Outer {  
    class Inner {  
        void display() {  
            System.out.println("No argument");  
        }  
  
        void display(String name) {  
            System.out.println(name);  
        }  
  
        void display(int age) {  
            System.out.println(age);  
        }  
    }  
}
```

These are overloaded methods:

```
display()  
display(String)  
display(int)
```

They have:

- same method name
 - different parameter lists
-

15. Inner classes: overriding

An inner class can also override an inherited method.

```
class Outer {  
  
    class Animal {  
  
        void sound() {  
            System.out.println("Animal sound");  
        }  
    }  
  
    class Dog extends Animal {  
  
        @Override  
        void sound() {  
            System.out.println("Bark");  
        }  
    }  
}
```

Here:

Dog.sound()

overrides:

Animal.sound()

The fact that the classes are inner classes does not fundamentally change overriding rules.

16. Static nested class vs inner class

Feature	Static Nested Class	Non-static Inner Class
Declared with static	Yes	No

Requires outer object for creation	No	Yes
Can overload methods	Yes	Yes
Can override methods	Yes	Yes
Can use lambdas	Yes	Yes
Can use method references	Yes	Yes
Can contain static methods	Yes, subject to normal class rules	Modern Java permits static members where language rules allow them; static initialization/member restrictions depend on Java version/context
Has implicit reference to outer instance	No	Yes

17. Exceptions and overloading

Exceptions have an important relationship with overloading.

Consider:

```
class Test {
    void process() throws IOException {
    }

    void process(int value) throws SQLException {
    }
}
```

This is valid because the methods have different parameter lists:

```
process()
process(int)
```

The exceptions do **not** make these methods overloaded.

This is very important.

18. Exceptions alone cannot overload a method

This is invalid:

```
class Test {  
  
    void process() throws IOException {  
    }  
  
    void process() throws SQLException {  
    }  
}
```

Why?

Because both methods have exactly the same signature:

```
process()
```

Changing only the throws clause does **not** constitute method overloading.

Rule

Return type and throws clause alone cannot distinguish overloaded methods.

19. What can distinguish overloaded methods?

Difference	Can create overload?
Different number of parameters	✓
Different parameter types	✓
Different parameter order	✓
Different parameter types/order	✓
Different return type only	✗
Different throws clause only	✗
Different access modifier only	✗
static vs non-static only	✗

Example:

```
void test(int x)
void test(String x)
```

Valid overloading.

But:

```
int test(int x)
double test(int x)
```

is invalid because only return type differs.

20. Exceptions and overriding

Exceptions are much more important for **overriding**.

Suppose:

```
class Parent {
    void process() throws IOException {
    }
}
```

A child class can override it with:

```
class Child extends Parent {
    @Override
    void process() throws FileNotFoundException {
    }
}
```

This is valid because:

```
FileNotFoundException
    ↓
IOException
```

FileNotFoundException is a narrower checked exception.

21. Overriding can omit the exception

This is also valid:

```
class Child extends Parent {  
  
    @Override  
    void process() {  
    }  
}
```

The overriding method can declare **fewer checked exceptions**, including none.

22. Overriding cannot broaden checked exceptions

This is invalid:

```
class Parent {  
  
    void process() throws IOException {  
    }  
}  
  
class Child extends Parent {  
  
    @Override  
    void process() throws Exception {  
    }  
}
```

Why?

Because:

```
Exception  
↑  
IOException
```

Exception is broader than IOException.

The subclass would allow callers to encounter a checked exception that the parent contract did not declare.

23. Overriding and unchecked exceptions

Unchecked exceptions are different.

```

class Parent {

    void process() throws IOException {
    }
}

class Child extends Parent {

    @Override
    void process() throws RuntimeException {
    }
}

```

This is allowed because RuntimeException is unchecked.

An overriding method may throw unchecked exceptions without being constrained by the checked-exception restriction.

24. Exception rules for overriding

Parent method	Child overriding method	Valid?	Reason
throws IOException	throws IOException	✓	Same checked exception
throws IOException	throws FileNotFoundException	✓	Narrower checked exception
throws IOException	no exception	✓	Can omit checked exceptions
throws IOException	throws Exception	✗	Broader checked exception
no checked exception	throws IOException	✗	Adds broader checked exception
no checked exception	throws RuntimeException	✓	Unchecked exception
throws Exception	throws IOException	✓	Narrower checked exception
throws Exception	throws RuntimeException	✓	Unchecked exception

--	--	--	--

25. Exceptions with lambda expressions

This is one of the most important practical areas.

Consider:

```
@FunctionalInterface
interface Processor {
    void process(String value) throws IOException;
}
```

A lambda can throw IOException:

```
Processor p =
    value -> {
        throw new IOException();
    };
```

This is valid because the functional-interface method declares:

throws IOException

26. Lambda cannot freely throw checked exceptions

Consider:

```
@FunctionalInterface
interface Processor {
    void process(String value);
}
```

This lambda is invalid:

```
Processor p =
    value -> {
        throw new IOException();
    };
```

Why?

Because the functional interface's abstract method does not declare IOException.

Important rule

A lambda's checked exceptions must be compatible with the throws clause of the functional interface's abstract method.

27. Lambda and exception compatibility

Functional interface method	Lambda throws	Valid?
void process() throws IOException	IOException	✓
void process() throws IOException	FileNotFoundException	✓
void process() throws IOException	no exception	✓
void process() throws IOException	Exception	✗
void process()	IOException	✗
void process()	RuntimeException	✓

This is conceptually similar to overriding because the lambda must satisfy the **contract of the functional interface's abstract method**.

28. Lambda and try-catch

If the functional interface does not permit the checked exception, handle it inside the lambda.

For example:

```
Consumer<String> consumer = value -> {  
    try {  
        readFile(value);  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
};
```

Here the lambda itself doesn't propagate IOException through the Consumer.accept() contract.

29. Method references and exceptions

Method references follow the exception compatibility rules of the referenced method and target functional interface.

Suppose:

```
class FileService {  
    static void read(String file) throws IOException {  
        System.out.println(file);  
    }  
}
```

And:

```
@FunctionalInterface  
interface FileProcessor {  
    void process(String file) throws IOException;  
}
```

Then:

```
FileProcessor processor =  
    FileService::read;
```

is valid.

Conceptually:

```
file -> FileService.read(file)
```

is valid for the same reason.

30. Method reference with incompatible checked exception

Suppose:

```
class FileService {  
    static void read(String file) throws IOException {  
    }  
}
```

But:

```
Consumer<String> consumer =  
    FileService::read;
```

This is invalid because:

```
Consumer.accept()
```

does not declare IOException.

The method reference cannot hide the checked-exception incompatibility.

31. Inner classes and exceptions

Inner-class methods follow the normal Java exception rules.

Overloading

```
class Outer {  
    class Inner {  
        void process(String value)  
            throws IOException {  
        }  
  
        void process(int value)  
            throws SQLException {  
        }  
    }  
}
```

This is valid.

Why?

Because the parameter lists differ:

```
process(String)  
process(int)
```

The different exceptions are **not** what makes them overloaded.

32. Inner class overriding with exceptions

```

class Outer {

    class Parent {

        void process()
            throws IOException {
        }
    }

    class Child extends Parent {

        @Override
        void process()
            throws FileNotFoundException {
        }
    }
}

```

This is valid because:

```

FileNotFoundException
↓
IOException

```

The normal overriding exception rules apply even though the classes are inner classes.

33. Complete comparison

Concept	Overloading	Overriding	Exception impact
Lambda	Lambda itself cannot be overloaded; overloaded methods accepting lambdas can create overload-resolution ambiguity.	Lambda itself does not perform traditional overriding; it implements a functional-interface method.	Checked exceptions must be compatible with the functional-interface abstract method.
Method reference	Method reference itself isn't overloaded; referenced methods may be overloaded and selected based on target type.	Referenced instance methods still follow normal overriding/dynamic dispatch.	Referenced method must be compatible with target functional-interface exception contract.

Inner class	Methods inside an inner class can be overloaded normally.	Inner classes can override inherited instance methods normally.	Normal Java checked/unchecked exception rules apply.
Static nested class	Methods can be overloaded.	Instance methods can be overridden.	Normal exception rules apply.
Anonymous class	Methods can be overloaded.	Methods can explicitly override superclass/interface methods.	Normal exception rules apply.
Exception declarations	throws clause alone cannot create an overload.	Checked exceptions can be narrowed/omitted, but not broadened.	Central to overriding compatibility.

34. Overloading vs overriding: master table

Parameter	Overloading	Overriding
Basic meaning	Same method name, different parameter list	Child implementation of inherited/implemented instance method
Requires inheritance?	No	Yes, normally
Same method name?	Yes	Yes
Same parameter list?	No	Yes
Return type	May differ if parameter list differs, but cannot be the only difference	Must be same or covariant
Access modifier	Can differ	Cannot reduce visibility
static	Static methods can be overloaded	Static methods are hidden, not overridden
final	Can overload a final method with different parameters	Final method cannot be overridden
private	Can overload	Private methods are not overridden

throws clause alone	Cannot create overload	Checked exceptions are restricted
Compile-time/runtime	Mainly compile-time	Method selection is runtime for overridden instance methods
Lambda itself	Not applicable directly	Not applicable directly
Inner-class methods	Yes	Yes
Method references	Can select among overloaded target methods	Can invoke dynamically dispatched overridden instance methods

35. Important example combining all four concepts

Consider:

```
import java.io.IOException;
import java.util.function.Consumer;

class Parent {

    void process(String value)
        throws IOException {
        System.out.println("Parent: " + value);
    }
}

class Child extends Parent {

    @Override
    void process(String value)
        throws IOException {
        System.out.println("Child: " + value);
    }

    // Overloading
    void process(int value) {
        System.out.println("Integer: " + value);
    }
}
```

Now:

```
Child child = new Child();
```

Method reference

```
Consumer<String> consumer =  
    child::process;
```

This references:

```
process(String)
```

not:

```
process(int)
```

because:

```
Consumer<String>
```

requires:

```
void accept(String)
```

36. Lambda equivalent

The method reference:

```
Consumer<String> consumer =  
    child::process;
```

is conceptually similar to:

```
Consumer<String> consumer =  
    value -> child.process(value);
```

But there is a problem:

`process(String)` declares:

throws `IOException`

while `Consumer<String>` does not.

Therefore this particular assignment is **not valid** unless the checked exception is handled appropriately.

A suitable functional interface would be:

```
@FunctionalInterface
```

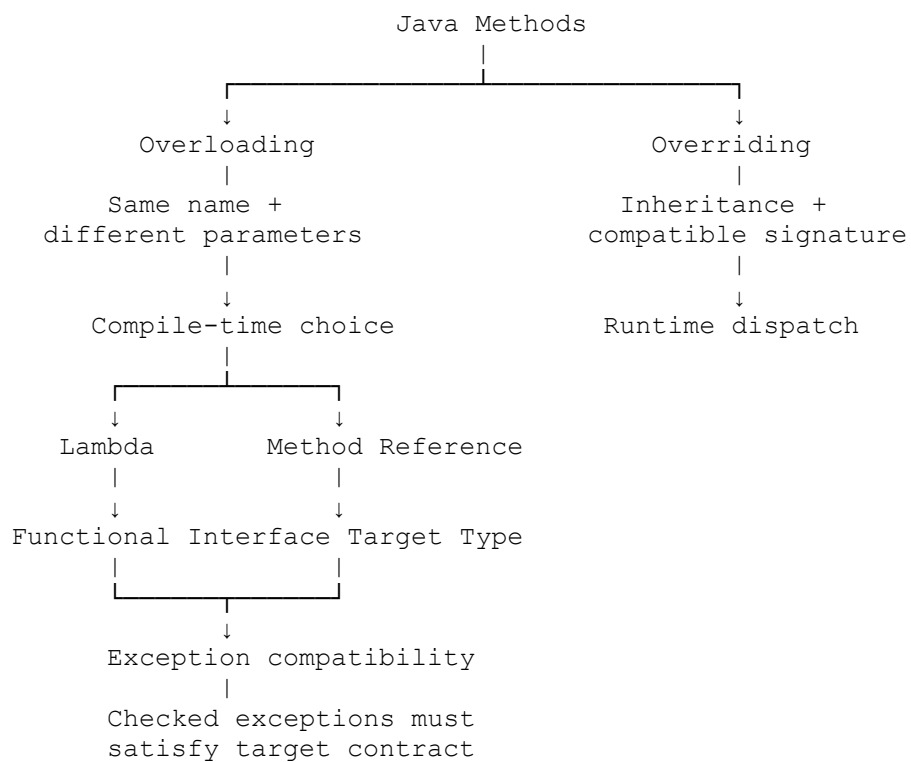
```
interface ThrowingConsumer<T> {
    void accept(T value) throws IOException;
}
```

Then:

```
ThrowingConsumer<String> consumer =
    child::process;
```

is valid.

37. Full relationship diagram



38. Interview-important rules

Rule	Explanation
1. Lambda is not a method declaration	Therefore you don't overload/override a lambda itself.
2. Lambda implements a functional interface	Its body provides the abstract method's behavior.

3. Method reference is not a method declaration	It refers to an existing method or constructor.
4. Method references can refer to overloaded methods	The target functional interface helps determine the appropriate overload.
5. Inner-class methods can be overloaded	Normal method-overloading rules apply.
6. Inner-class methods can be overridden	Normal inheritance/overriding rules apply.
7. throws does not create overloading	process() throws IOException and process() throws SQLException cannot coexist as overloads.
8. Checked exceptions matter during overriding	An overriding method cannot declare a broader checked exception.
9. Lambda checked exceptions depend on the functional interface	The lambda cannot arbitrarily introduce a checked exception.
10. Method references follow the same exception compatibility rules	The referenced method must be compatible with the target functional interface.
11. Unchecked exceptions are less restrictive	RuntimeException and its subclasses do not need to be declared.
12. Static methods are hidden, not overridden	This applies even when classes are nested/inner classes.

39. Final summary table

Feature	Lambda Expression	Method Reference	Inner Class	Exception
Can be overloaded directly?	✗ No	✗ No	✓ Methods can	throws alone ✗
Can participate in overloaded method calls?	✓	✓	✓	✓ indirectly
Can be overridden directly?	✗ No	✗ No	✓ Methods can	✓ affects overriding

Can implement interface behavior?	✓	✓	✓	Subject to method contract
Can use static methods?	✓	✓	✓	Normal rules
Can use instance methods?	✓	✓	✓	Normal rules
Can use inner-class methods?	✓	✓	✓	Normal rules
Can reference overridden instance method?	✓ via invocation in lambda	✓	✓	Normal rules
Checked exception allowed automatically?	Only if target interface permits it	Only if target interface permits it	According to method declaration	Must follow Java exception rules
throws alone creates overload?	✗	✗	✗	✗
Can narrow checked exception while overriding?	Target contract permitting	Target contract permitting	✓	✓
Can broaden checked exception while overriding?	✗	✗	✗	✗

The simplest way to remember

Lambda = implementation of a functional interface.

Method reference = reference to an existing method/constructor.

Inner-class methods = normal Java methods, so they can be overloaded and overridden.

Exceptions = throws cannot create an overload, and overriding methods cannot broaden checked exceptions.

The particularly important interview distinction is:

```
// Overloading
void test(int x);
void test(String x);    // VALID
```

```
// Exception does NOT create overloading
void test() throws IOException;
void test() throws SQLException; // INVALID
```

and:

```
// Overriding — narrower checked exception is allowed
class Parent {
    void test() throws IOException {}
}

class Child extends Parent {
    @Override
    void test() throws FileNotFoundException {}
}
```

while:

```
class Child extends Parent {
    @Override
    void test() throws Exception {} // INVALID
}
```

because Exception is broader than IOException.